Generating a Phylogenetic Tree Using Pairwise MASH Distances and Neighbor Joining Algorithm

Project Report

Aditya Kamat | Devansh Gupta

Overview of the problem

A core challenge in comparative genomic analysis is to determine genomic similarity and evolutionary relationships between organisms. While multiple sequence alignment (MSA) is commonly used to construct phylogenetic trees, such methods are computationally intensive and do not scale well to large datasets. The pairwise Mash distance problem addresses this by using MinHash sketching (dimensionality reduction) to estimate genomic similarity. By hashing a large genome sequence into a smaller *sketch*, the Jaccard index J between sketches is used to estimate the similarity between a pair of sequences.

Using J, the Mash distance between two sequences is computed as

$$D = -\frac{1}{k} \ln\left(\frac{2J}{1+J}\right),$$

where k is the k-mer size. The resulting $N \times N$ distance matrix can then be used to build a phylogenetic tree. Here N is the number of sequences to be compared.

Compared with MSA, Mash is substantially more scalable: MSA has time and memory complexity $O(N^2L^2)$, while Mash has time complexity $O(NL)$ and memory complexity $O(NS + N^2)$, where S is sketch size. Hence, Mash is an important method when working with large genome datasets, with applications such as whole-sequence comparisons, fast similarity screening, and large-scale genome clustering. It is also commonly used as a preprocessing step to select genomes for deeper analysis.

Current State-of-the-Art

Prior work and current state-of-the-art methods for sketch-based genomic comparison include the following:

- **Dashing:** Uses an alternating hashing strategy based on HyperLogLog.
- **SKmer:** Uses error-corrected sketching for genomes.
- **DIPPER[1]:** Uses different strategies (including placement and divide-and-conquer style methods) to generate phylogeny depending on dataset scale.

Our baseline implementation remains centered on standard MinHash + Neighbor-Joining, with additional validation against DIPPER reference trees.

Other state-of-the-art tools are informative but are not used as direct baselines because they modify the underlying method rather than parallelizing standard MinHash. Because our project specifically targets parallelization of the original MinHash workflow, our sequential baseline code remains the most direct baseline for quantifying speedup while preserving algorithmic equivalence.

Methodology

Our entire algorithm methodology has been divided into three phases as also shown in Figure 1.:

1. Generate a MASH sketch for each genomic sequence in the dataset. Let the set of k-mers from a sequence be
$$K = \{x_1, x_2, \ldots, x_m\}.$$
For each k-mer, we use the same GPU hash pipeline as our implementation: MurmurHash3 x64_128 (seed `HASH_SEED=1337`) and return the 64-bit output h_1. For one 16-byte block, the core updates are
$$k_1 \leftarrow k_1 c_1, \quad k_1 \leftarrow \mathrm{ROTL}_{64}(k_1, 31), \quad k_1 \leftarrow k_1 c_2, \quad h_1 \leftarrow h_1 \oplus k_1,$$
$$h_1 \leftarrow \mathrm{ROTL}_{64}(h_1, 27), \quad h_1 \leftarrow h_1 + h_2, \quad h_1 \leftarrow 5h_1 + \texttt{0x52dce729},$$
with the analogous update for k_2, h_2 using constants $c_1 =$ `0x87c37b91114253d5` and $c_2 =$ `0x4cf5ad432745937f`. Tail bytes are folded by a switch over `len & 15`. Finalization applies length xor, cross-sum ($h_1 += h_2,\ h_2 += h_1$), and avalanche:
$$\mathrm{fmix64}(z): \ z \leftarrow z \oplus (z \gg 33), \quad z \leftarrow z \cdot \texttt{0xff51afd7ed558ccd}, \quad z \leftarrow z \oplus (z \gg 33),$$
$$z \leftarrow z \cdot \texttt{0xc4ceb9fe1a85ec53}, \quad z \leftarrow z \oplus (z \gg 33).$$
After hashing all k-mers, we sort hashes on GPU (`thrust::sort`) and define the sketch as the smallest `bottomK` values:
$$S = \{h_{(1)}, h_{(2)}, \ldots, h_{(b)}\}, \quad b = \min(m, \texttt{bottomK}).$$

2. For each pair of sequences (i, j), compute Jaccard similarity, Mash distance, and populate the distance matrix. Let the sorted sketches be S_i and S_j, each of size at most $b =$ `bottomK`. We compute the intersection count using a parallel merge-style traversal (one sequence pair per GPU block):
$$c_{ij} = |S_i \cap S_j|.$$
Then compute union size and Jaccard index as
$$u_{ij} = |S_i \cup S_j| = |S_i| + |S_j| - c_{ij}, \qquad J_{ij} = \frac{c_{ij}}{u_{ij}}.$$
Using J_{ij}, Mash distance is
$$D_{ij} = -\frac{1}{k} \ln\left(\frac{2J_{ij}}{1 + J_{ij}}\right),$$
where $k =$ `kmerSize`. We write D_{ij} into the symmetric matrix $\mathbf{D}$ as $\mathbf{D}[i, j] = \mathbf{D}[j, i] = D_{ij}$ and set diagonal entries $\mathbf{D}[i, i] = 0$.

3. Generate a phylogenetic tree from the distance matrix using the Neighbor-Joining (NJ) algorithm [5]. For the current active node set of size n, first compute net divergence for each node:

$$r_i = \sum_{k \neq i} D_{ik}.$$

Then form the NJ selection matrix

$$Q_{ij} = (n-2)D_{ij} - r_i - r_j, \qquad i \neq j,$$

and select the pair (u, v) with minimum Q_{ij} as the next neighbors to join. The branch lengths from the new internal node x to u and v are

$$\delta_{u,x} = \frac{1}{2}D_{uv} + \frac{r_u - r_v}{2(n-2)}, \qquad \delta_{v,x} = D_{uv} - \delta_{u,x}.$$

After joining (u, v), update distances from x to every remaining node k as

$$D_{xk} = \frac{D_{uk} + D_{vk} - D_{uv}}{2}.$$

Remove nodes u and v, insert the newly formed node x, and repeat this process until only two nodes remain; connect this final pair with an edge length of $D_{ab}/2$. In our implementation, we parallelize the r_i computation, Q-matrix construction, minimum-Q reduction, and distance-matrix updates during each iteration.

The generated phylogenetic tree is compared with the baseline ground truth using the Robinson Fould (RF) score metric.

How the Algorithm will be Parallelized

Platform: CPU(Intel TBB) and GPU (CUDA).

Algorithm steps:

1. **Generate Mash sketches (for all sequences):** map parallelism across sequences/blocks.
   (a) Compress genomic sequences with binary encoding: map parallelism across threads using TBB::parallel_for.
   (b) Create k-mers of a given sequence: map parallelism across threads within a block.
   (c) Hash the k-mers: map parallelism.
   (d) Sort hashed k-mers: parallel sort.
   (e) Retain the bottom n hashes (the sketch).
2. **Compute pairwise distances between sketches A_s and B_s:** map parallelism across GPU blocks (one pair of sequences per block).
   (a) Compute $|A_s \cap B_s|$: map parallelism across threads in a block.
   (b) Compute $|A_s \cup B_s|$ via $|A_s \cup B_s| = |A_s| + |B_s| - |A_s \cap B_s|$.

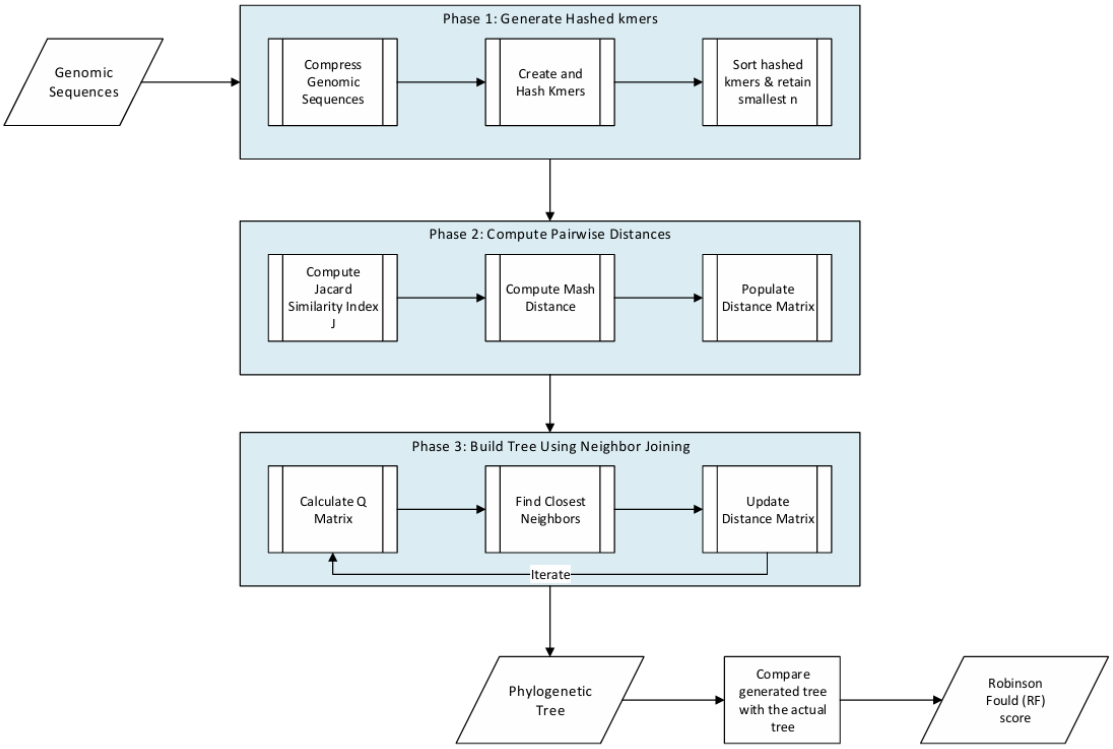


Figure 1: End-to-end Mash + Neighbor-Joining methodology pipeline

(c) Compute Jaccard index $J = \frac{|A_s \cap B_s|}{|A_s \cup B_s|}$: map parallelism.

(d) Compute Mash distance D: map parallelism across threads in a block.

(e) Populate the pairwise distance matrix.

3. **Build a tree using the Neighbor-Joining algorithm:**

   (a) Calculate the Q-matrix: map parallelism.

   (b) Find closest neighbors (minimum in Q): parallel min-reduction.

   (c) Update the distance matrix: map parallelism.

Evaluation Metrics

To evaluate performance and biological fidelity, we use the DIPPER benchmark subset containing 10,000 genomic sequences and compare generated trees against the corresponding reference trees available in DIPPER dataset.

- We compare our parallel implementation against our sequential baseline implementation and report speedup achieved for each major kernel, as well as end-to-end runtime improvement.
- We compare generated trees with the reference tree available in DIPPER dataset and use the normalized Robinson–Foulds (RF) score[6] to quantify phylogenetic accuracy.
- We analyze the impact of varying `kmerSize` and `bottomK` on tree accuracy.
- We analyze the impact of varying `bottomK` on runtime.

Results

On running the algorithm iteratively over the range `kmerSize` $\in [5, 32]$ and `bottomK` $\in [1000, 8000]$ with a step size of 1000, the lowest robinson fould score in comparison to the ground truth tree was observed for the configuration `(kmerSize,bottomK)` $= (11, 8000)$ with a robinson fould score of 0.23 compared to the ground truth tree which is comparable to the results obtained in DIPPER for 10000 sequences. This, along with the fact that our observations corroborate the expected trends, verifies the correctness of our implementation.

On a run with 1,000 sequences (49,995,000 pairwise comparisons) on an NVIDIA A30 GPU with `kmerSize=11` and `bottomK=8000`, we observe large kernel-level speedups (Table 1).

Table 1: GPU kernel execution time comparison between sequential and parallel implementations for `kmerSize=11` and `bottomK=8000` on 1,000 sequences.

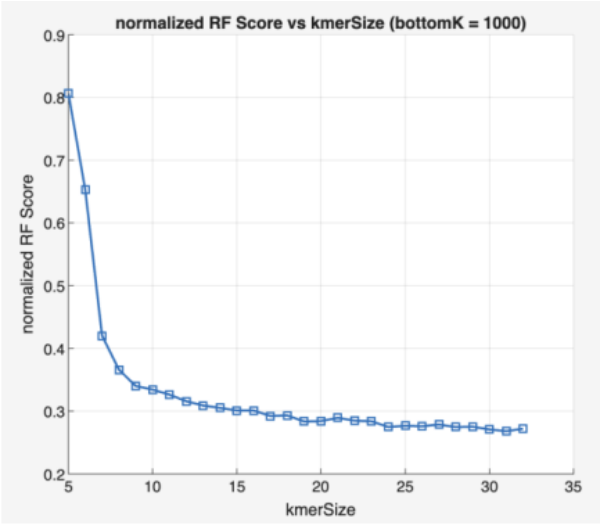
Kernel	**Sequential Total (ns)**	**Parallel Total (ns)**	**Speedup**
ComputeMashDistancesKernel	639,348,354,311	217,173,979	2943.95×
FindMinQ + FinalizeTopology	23,273,376,567	218,445,715	106.54×
ComputeNetDivergence_Kernel	20,224,370,357	72,911,274	277.38×
HashAllKmersKernel	4,251,872,296	3,948,922	1076.72×
kmerArrCreate	542,113,006	5,163,873	104.98×
UpdateDistanceMatrix_Kernel	148,602,157	2,437,754	60.96×
InitNJMatrixKernel	39,851,588	44,416	897.23×
Total	687,828,540,282	520,125,933	**1322.43**×

It was observed that on keeping bottomK constant and varying kmerSize, the accuracy of the generated trees improves intially and then saturates beyond a point as we expect. Further it was noticed that the normalized RF score follows a decreasing trend on increasing bottomK while keeping kmerSize constant. This is expected as on increasing bottomK, the algorithm has a larger sketch size to find differences between the kmers of a pair of sequences.

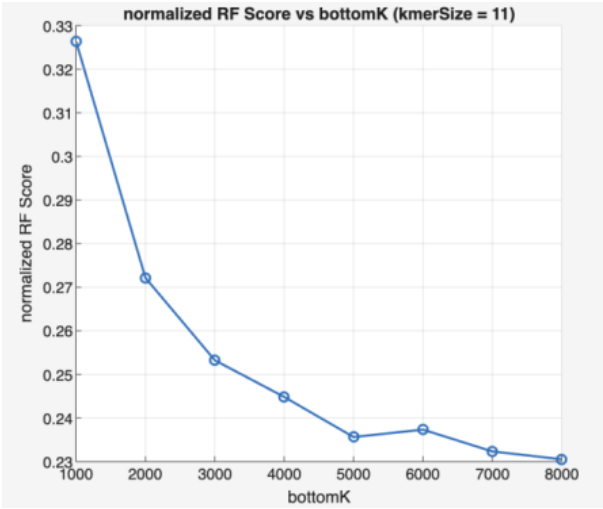
The runtime for the ComputeMashDistances kernel is expected to scale linearly with bottomK. However we see a slight inflection beyond bottomK=3000, which is attributed to the way our algorithm allocates shared memory and invokes streaming multiprocessors (SMs). Beyond bottomK=3000, the shared memory allocated exceeds the 48KB limit and hence we have to drop one SM which leads to a constant overhead added to the runtimes of all cases beyond bottomK=3000. Barring this constant overhead, the graph scales linearly.

Finally we varied the number of sequences read and compared with the afforementioned optimal configuration and observed the ComputeMashDistances kernel to vary quadratically with the number of sequences as expected.

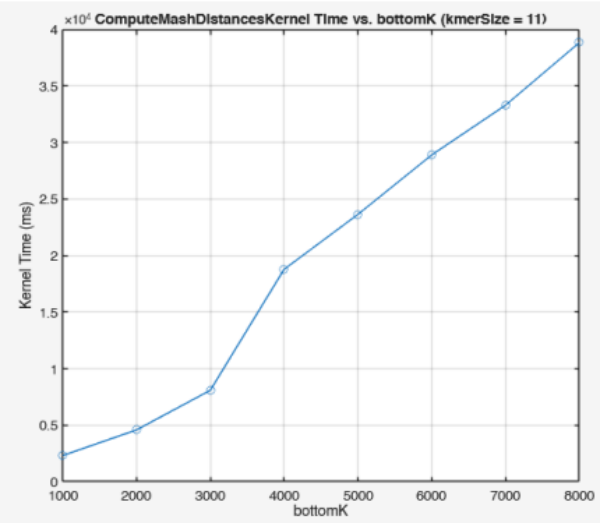
The complete codebase for this project is available at github.com/devansh-gupta19/ECE213-Phylogenetic-Tree-Using-Mash.



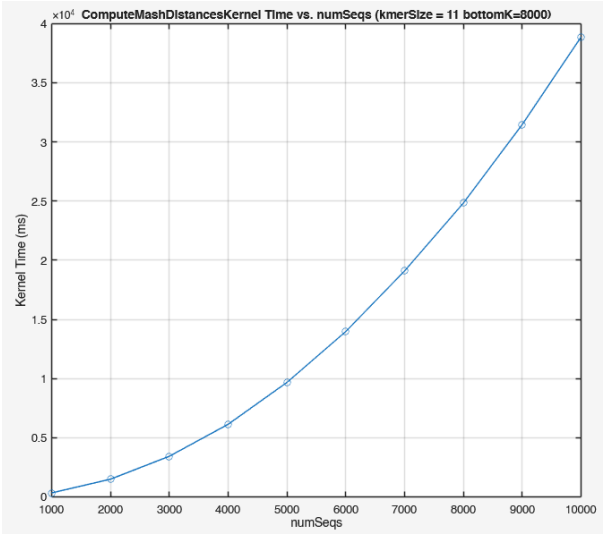
(a) Effect of varying `kmerSize` on tree accuracy.



(b) Effect of varying `bottomK` on tree accuracy.



(c) Runtime trend versus `bottomK`.



(d) Runtime trend versus `numSeqs`.

Figure 2: Observations: accuracy trends (a,b) and runtime trends (c,d).

References

1. Sumit Walia, Zexing Chen, Yu-Hsiang Tseng, Yatish Turakhia, "Ultrafast and Ultralarge Distance-Based Phylogenetics Using DIPPER," bioRxiv, 2025. DOI: 10.1101/2025.08.12.669583.

2. Ondov, Brian D., et al. *Mash: Fast Genome and Metagenome Distance Estimation Using MinHash*. GitHub, https://github.com/marbl/Mash.

3. Broder, A.Z. On the resemblance and containment of documents. Compression and Complexity of Sequences 1997 - Proceedings, 21-29 (1998).

4. Fan, H., Ives, A.R., Surget-Groba, Y. & Cannon, C.H. An assembly and alignment-free method

of phylogeny reconstruction from next-generation sequencing data. BMC genomics 16, 522 (2015).

5. Wikipedia contributors. "Neighbor joining." Wikipedia, The Free Encyclopedia. Wikipedia, The Free Encyclopedia, 21 Aug. 2025. Web. 14 Mar. 2026.

6. Wikipedia contributors. "Robinson–Foulds metric." Wikipedia, The Free Encyclopedia. Wikipedia, The Free Encyclopedia, 14 Jan. 2026. Web. 14 Mar. 2026.

AI Use Declaration

1. Used during literature review to summarize research papers.

2. Used to generate script to compare our generated tree with the reference tree and compute Robinson-Fould (RF) score.

3. Used to generate MATLAB script to generate graphs.

4. Used to edit runCommands.sh script to perform parameter sweep for kmerSize and bottomk.

5. Used to help with creating newickHelper.cpp to create .nwk files from the generated trees.